

SecurAx Scanner Engine

Manuel Analytique Complet - 11 Scanners

*Notre philosophie : Autocritique constante.
Identifier les limites avec honnetete, puis proposer des ameliorations concretes pour
les depasser.*

Base Documentaire	Analyse du code source reel + V3 PDF + SCANNERS_TECHNICAL.md
Couverture	11 scanners (vs 6 dans la V3)
Architecture Cible	Cloud dedie, sans contraintes d'hebergement
Date	Juin 2026
Version	V4 - Edition Complete

Table des Matieres

GROUPE A - Scanners Applicatifs

- | | | |
|----|-----------------------------|---------------------------|
| 1. | Scanner Web Application | <i>web_scanner.py</i> |
| 2. | Scanner DAST Multi-Moteurs | <i>dast_scanner.py</i> |
| 3. | Scanner SAST Multi-Langages | <i>sast_scanner.py</i> |
| 4. | Scanner Reseau & Ports | <i>netscan_scanner.py</i> |

GROUPE B - Scanners Infrastructure

- | | | |
|----|-------------------------------|--------------------------|
| 5. | Scanner SSL / TLS | <i>ssl_scanner.py</i> |
| 6. | Scanner de Dependances | <i>dep_scanner.py</i> |
| 7. | Scanner Configuration Serveur | <i>server_scanner.py</i> |
| 8. | Scanner Serveur Externe | <i>server_ext.py</i> |

GROUPE C - Scanners Specialises

- | | | |
|-----|---------------------|-----------------------------|
| 9. | Scanner Docker | <i>docker_scanner.py</i> |
| 10. | Scanner DNS & Email | <i>dns_scanner.py</i> |
| 11. | Scanner WordPress | <i>wordpress_scanner.py</i> |

Structure par scanner (sections 1 a 11) :

X.1 Capacites & Perimetre d'Action | X.2 Limites Reelles & Autocritique | X.3 Ameliorations Proposees

1. SCANNER WEB APPLICATION

Fichier source : [web_scanner.py](#)

Audit passif de la couche HTTP base sur une requete initiale (GET). Analyse les headers de securite, l'empreinte serveur, la configuration des cookies et les methodes HTTP exposees.

1.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Headers securite	Verifie HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy et Permissions-Policy.
Empreinte serveur	Analyse Server, X-Powered-By et X-Generator pour reveler la stack technique cote serveur.
Redirection HTTPS	Verifie si HTTP redirige vers HTTPS (301/302) et si HSTS preload est correctement active.
Cookies	Controle les attributs Secure, HttpOnly, SameSite et le prefixe __Host- pour une isolation maximale.
Methodes HTTP	Detecte les methodes dangereuses activees via OPTIONS : PUT, DELETE, TRACE, CONNECT.

1.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de crawling	Ne verifie que la racine '/', ignorant 99% du contenu reel du site.
Aucun test actif	Zero payload envoye - ne detecte pas les failles XSS ou SQLi reelles.
Pas de CORS check	Ne teste pas les politiques CORS permissives (Access-Control-Allow-Origin: *).
Pas d'enumeration	Ne teste pas les chemins sensibles courants (/admin, /.git, /backup, /.env).
Cache/CDN opaque	Un CDN peut masquer la vraie configuration du serveur d'origine derriere le cache.

1.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Crawler léger (BeautifulSoup)	Extraire 10-20 sous-pages pour verifier la coherence des headers sur tout le site. Priorite Haute.

Test CORS actif	Envoyer des requetes avec Origin: https://evil.com pour capturer les configs CORS permissives. Priorite Haute.
Audit robots.txt & sitemap	Analyser robots.txt et sitemap.xml pour reveler la structure et les chemins sensibles exposes. Priorite Moyenne.
Enumeration de chemins	Tester /admin, /.git, /.env, /backup via une wordlist de 500 entrees. Priorite Moyenne.
Detection WAF	Fingerprinting des WAF via headers caracteristiques et reponses a des payloads inoffensifs. Priorite Basse.

2. SCANNER DAST MULTI-MOTEURS

Fichier source : `dast_scanner.py`

Orchestrateur d'analyse dynamique utilisant 3 moteurs externes (OWASP ZAP, Nuclei, Nikto) et un moteur natif de secours.
Approche boîte noire : déduit les vulnérabilités depuis le comportement HTTP observable.

2.1 Capacités et Périmètre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalité	Détail de l'analyse
OWASP ZAP	Demon + API REST - spidering actif + injections XSS/SQLi/CSRF automatisées avec rapport détaillé.
Nuclei	CLI/Cloud - plus de 9 000 templates YAML orientés CVE et expositions publiques connues.
Nikto	Vérification des configurations serveurs web et détection de fichiers obsolètes ou dangereux.
Fallback Natif	Utilise requests avec des payloads basiques si les moteurs externes sont inactifs ou absents.

2.2 Limites Réelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Temps d'exécution massif	ZAP + Nuclei peuvent prendre 15 à 30 minutes sur un site moyen.
Authentification complexe	Incapable de scanner derrière des portails SSO ou MFA sans Selenium/Playwright.
Blocage WAF	Le trafic massif est souvent bloqué comme une attaque DDoS, aveuglant le scanner et produisant des faux négatifs.
Approche Boîte Noire	Déduit les failles uniquement via le comportement HTTP, sans voir le code source.
GraphQL non couvert	Les endpoints GraphQL non-REST échappent au spidering standard de ZAP/Nikto.

2.3 Améliorations Proposées pour l'Architecture Cible

Axes de R&D concrets pour dépasser les limites actuelles :

Axe d'Amélioration	Description Technologique / Impact
Support Auth Bearer/JWT	Accepter des tokens de session et les injecter dynamiquement pour scanner les zones authentifiées. Priorité Haute.
Throttle intelligent	Auto-ralentissement si réponse 429 - backoff exponentiel pour éviter le bannissement IP. Priorité Haute.

Scanner GraphQL	Detecter les endpoints GraphQL et tester l'introspection et les injections specifiques. Priorite Moyenne.
Test JWT	Tester : alg:none, secret HS256 faible, expiration ignoree. Priorite Moyenne.
Vision Produit - Agent IAST Optionnel	Module agent installable par le client pour corroler l'attaque DAST avec la ligne de code exacte. Requiert accord contractuel explicite. Long terme.

3. SCANNER SAST MULTI-LANGAGES

Fichier source : `sast_scanner.py`

Analyse statique concurrente de code source via 4 moteurs dedies (Bandit, Semgrep, Gitleaks, Natif). Detecte les vulnerabilites dans le code, les secrets hardcodes et les patterns dangereux sans executer le programme.

3.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Bandit (Python)	Linter securite : detecte eval(), exec(), subprocess, injections SQL et deserialisation non securisee.
Semgrep (Polyglotte)	Analyse AST multi-langages avec regles OWASP Top 10 et CWE Top 25. Supporte Python, JS, Java, Go.
Gitleaks	Detection de secrets hardcodes : cles AWS, tokens GitHub, Stripe, mots de passe en clair.
Scanner Natif	Fallback regex ciblant les patterns critiques sans dependances systeme externes.

3.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Faux positifs structurels	Alarme sur du code mort (dans /tests/) ou des variables mockees ('test_password').
Manque de Taint Tracking	Analyse par fichier : ne suit pas le flux de donnees entre plusieurs modules ou microservices.
Contexte d'execution ignore	Signale des failles pouvant etre mitigees par des WAF ou des wrappers globaux.
Secrets hors code ignores	Gitleaks scanne le code committe mais pas les fichiers .env non versionnees.
Dependances transitives	Ne resout pas l'arbre complet des imports entre fichiers et modules.

3.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Filtres IA (ARIA)	Envoyer les resultats bruts au moteur IA pour evaluer la pertinence reelle et eliminer les faux positifs. Deja disponible dans SecurAx.
Taint Analysis (CodeQL)	Integrer CodeQL pour un suivi complet : entree utilisateur -> execution base de donnees. Priorite Haute.

Scan Diff & .gitignore	Ne scanner que les fichiers modifies via 'git diff' pour une integration CI/CD ultrarapide. Priorite Haute.
Analyse fichiers .env	Etendre Gitleaks aux fichiers d'environnement non commites via upload direct securise. Priorite Moyenne.
Detection Dependency Confusion	Verifier si les noms de packages internes existent sur PyPI/npm (risque supply chain). Priorite Moyenne.

4. SCANNER RESEAU & PORTS

Fichier source : `netscan_scanner.py`

Cartographie de la surface d'attaque reseau externe. Decouverte des services exposes, fingerprinting OS et bannieres logicielles. Requier des privileges adaptes pour un scan SYN optimal (root ou NET_RAW capability).

4.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Scan TCP (Nmap)	Decouverte des services exposes avec un scan furtif SYN Half-open. Plage de ports configurable par l'utilisateur.
Fingerprinting	Detection precise des OS et bannieres logicielles via les options Nmap -sV -O.
Scripts NSE	Execution de scripts Nmap pour valider des vulnerabilites connues (CVE) sur les services detectes.
Banner Grabbing	Extraction de la version exacte des services pour croisement avec la base CVE.

4.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Bloque par IDS/IPS	Les scans massifs declenchent les pare-feu et systemes de prevention d'intrusion (Snort, Suricata).
Pas de contexte applicatif	Sait que le port 80 est ouvert mais n'inspecte pas l'arborescence web ni les services derriere.
Requier root/raw sockets	Pour le scan SYN Half-open optimal. Sans root : scan TCP connect degrade uniquement.
UDP ignore	Services DNS (53), SNMP (161), NTP (123), IPSec (500) sur UDP non couverts par default.
Pas d'intelligence IP	Ne croise pas l'IP avec les donnees BGP, ASN ou les listes de reputation (listes noires).

4.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
--------------------	------------------------------------

Scan Natif Multi-thread	Implementer socket.connect_ex() en parallele comme fallback performant sans root. Priorite Haute.
Timing evasif (-T2)	Espacer dynamiquement les requetes pour passer sous le radar des IDS actifs. Priorite Haute.
Rotation IP - Pool Dedie	Proxy pool propriétaire (NON Tor) comme Qualys/Tenable - legal, stable, sans risque de bannissement. Priorite Moyenne.
Scan UDP partiel	Cibler les ports UDP critiques : 53 (DNS), 161 (SNMP), 123 (NTP), 500 (IPSec). Priorite Moyenne.
Intelligence IP (ASN/Reputation)	Croiser avec Shodan/WHOIS pour le contexte d'exposition et les listes noires publiques. Priorite Basse.

5. SCANNER SSL / TLS

Fichier source : [ssl_scanner.py](#)

Audit cryptographique de la couche de transport. Verifie l'expiration, les protocoles deprecies, les algorithmes faibles et la validite de la chaine de certificats X.509.

5.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Expiration	Alerte si le certificat expire dans moins de 30 jours. Seuil configurable.
Protocoles	Detecte SSLv2/v3 et TLS 1.0/1.1 deprecies (RFC 8996 - mars 2021).
Ciphers faibles	Verifie l'absence de Forward Secrecy et les algorithmes vulnerables : RC4, DES, NULL, EXPORT.
Parsing ASN.1	Extrait SAN, issuer, subject et dates directement du certificat X.509.
Chain validation	Verifie la chaine de certificats jusqu'au CA racine de confiance.

5.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Port 443 uniquement	Ignore les autres ports TLS : 8443, IMAPS (993), SMTPS (465), LDAPS (636), PostgreSQL (5432).
Pas de POODLE/Heartbleed	Ces failles requierent des handshakes volontairement malformes que la librairie ssl native Python ne permet pas.
Pas de verification CT	Ne verifie pas si le certificat est present dans les logs de Certificate Transparency.
Pas d'OCSP	Ne verifie pas si le certificat est revoque via OCSP ou CRL (Certificate Revocation List).
Pas de test HTTP/2	Ne detecte pas les problemes lies a ALPN et a la negociation de protocole HTTP/2 ou HTTP/3.

5.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
--------------------	------------------------------------

Integration SSLyze	Remplacer la lib ssl native par SSLyze (pure Python) pour detecter ROBOT, BEAST, POODLE, Heartbleed. Priorite Haute.
CT Log Check (crt.sh API)	Interroger crt.sh pour decouvrir des sous-domaines caches lies au certificat principal. Priorite Haute.
Multi-Ports TLS	Coupler avec le Network Scanner pour verifier TLS sur l'ensemble des ports ouverts. Priorite Moyenne.
Verification OCSP	Verifier la revocation du certificat en temps reel via OCSP stapling. Priorite Moyenne.
Score Mozilla Observatory	Integrer le scoring SSL de Mozilla Observatory pour un rapport standardise et comparable. Priorite Basse.

6. SCANNER DE DEPENDANCES

Fichier source : `dep_scanner.py`

Supply Chain Analysis via l'API Google OSV.dev. Parse les fichiers de dependances (requirements.txt, package.json, composer.json), extrait les versions exactes et croise avec la base CVE en temps reel.

6.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
SCA & SBOM	Parse requirements.txt, package.json, composer.json et extrait les versions exactes de chaque paquet.
Mapping CVE	Croise l'inventaire avec OSV.dev en temps reel pour chaque version declaree.
Detection de malwares	Repere les dependances obsoletes, abandonnees ou potentiellement typosquatees.

6.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Faux positifs structurels	Signale la librairie entiere comme vulnerable, meme si la fonction affectee n'est pas appelee.
Dependances transitives	Sans fichier lock strict (poetry.lock, yarn.lock), les sous-dependances critiques sont ignorees.
Rate Limiting API OSV	Des centaines de requetes individuelles peuvent saturer et ralentir l'API OSV.dev.
Source unique	OSV.dev peut avoir des delais sur les CVE recents versus NVD ou GitHub Advisory Database.
Pas de controle de licence	Ne detecte pas les conflits de licences GPL dans les projets proprietaires.

6.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Batch OSV API	Envoyer les packages par lots de 100 via une seule requete POST - reduction de 90% du temps d'execution. Priorite Haute.
Generation SBOM (Syft)	Utiliser Syft pour resoudre l'arbre de dependances complet meme sans fichier lock. Priorite Haute.

Reachability Analysis	Croiser avec l'AST Semgrep pour confirmer si le code vulnérable est effectivement appelé. Priorité Haute.
Source Secondaire NVD	Fallback API NVD pour les CVE non encore indexés sur OSV. Priorité Moyenne.
Contrôle de Licences	Détecter les conflits : GPL dans un projet propriétaire. Priorité Basse.

7. SCANNER CONFIGURATION SERVEUR

Fichier source : `server_scanner.py`

Audit de la configuration des serveurs web (Apache, Nginx) via l'analyse des headers HTTP exposes et des fichiers accessibles. Detecte les mauvaises pratiques de durcissement (hardening).

7.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Directives serveur	Verifie ServerTokens, ServerSignature, autoindex via les headers de reponse HTTP.
Fichiers sensibles exposes	Detecte l'accessibilite publique de .htaccess, .env, .git, web.config, phpinfo.php.
Headers de securite	Croise avec le scanner web pour coherence de la politique de securite globale.
Modules dangereux	Detecte mod_status, mod_info actives sans authentication (Apache).
Version disclosure	Repere la divulgation de version dans les pages d'erreur (404, 500).

7.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Vue externe uniquement	Ne peut inspecter que ce qui est expose via HTTP - pas les fichiers de config internes (/etc/nginx/).
Pas de baseline CIS	Ne compare pas la config contre les benchmarks CIS Apache/Nginx officiels.
Pas de detection backup	Ne cherche pas systematiquement les fichiers de backup (.bak, .old, .orig, ~).
Pas d'auth config	Ne verifie pas la robustesse des zones protegees par Basic Auth ou Digest Auth.

7.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
CIS Benchmark Mapping	Croiser les resultats avec le CIS Apache/Nginx Benchmark pour un score de conformite officiel. Priorite Haute.
Detection Backup Files	Tester .bak, .old, .orig, ~, .swp sur les configs et fichiers connus. Priorite Haute.
Test Directory Listing	Verifier si l'autoindex est actif sur les repertoires sensibles. Priorite Moyenne.

Integration SCAP	Support du protocole SCAP pour echange standardise des resultats avec les SIEMs d'entreprise. Priorite Basse.
------------------	---

8. SCANNER SERVEUR EXTERNE

Fichier source : `server_ext.py`

Reconnaissance passive de la surface d'attaque reseau depuis une perspective externe. Exploite des donnees OSINT (Shodan, headers HTTP) sans envoyer de scan actif vers la cible.

8.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Integration Shodan	Interroge l'API Shodan : ports decouverts, services, bannieres et CVE connus pour l'IP cible.
Headers HTTP externes	Analyse les headers de reponse depuis la perspective d'un client externe.
Reputation IP	Verifie si l'IP cible figure dans des listes noires (spam, malware, botnet).
Correlation ports/services	Croise les ports ouverts avec les services attendus pour detecter les expositions anormales.

8.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Depend de Shodan	Les resultats sont lies a la frequence de scan de Shodan - potentiellement obsoletes de plusieurs jours.
Pas de scan actif	Ne sonde pas directement les ports - se base sur les donnees Shodan indexees.
Rate limiting Shodan	L'API gratuite est tres limitee en nombre de requetes par mois.
Vue externe uniquement	Ne voit pas la surface d'attaque interne ni les services sur des plages IP privees (RFC 1918).

8.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Source Secondaire Censys	Completer Shodan avec Censys.io pour des donnees plus fraiches et exhaustives. Priorite Haute.
Scan Actif Complementaire	<code>socket.connect_ex()</code> sur ports critiques pour verification en temps reel. Priorite Haute.
Threat Intelligence	Croiser l'IP avec VirusTotal et AlienVault OTX pour le contexte menace complet. Priorite Moyenne.

Historique DNS Passif	Comparer DNS actuel vs historique PassiveDNS pour detecter les changements suspects recents. Priorite Basse.
-----------------------	--

9. SCANNER DOCKER

Fichier source : `docker_scanner.py`

Analyse statique des images Docker : detection des mauvaises pratiques dans le Dockerfile, des secrets hardcodes et des packages systeme vulnerables via croisement CVE.

9.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Analyse Dockerfile	Detecte : user root, ports exposes inutiles, secrets dans ENV/ARG, absence de multi-stage build.
CVE Mapping packages	Croise les packages systeme installes dans l'image avec OSV/NVD pour les CVE connus.
Layer Analysis	Inspecte chaque layer Docker pour detecter des secrets introduits puis supprime (persistants dans l'historique).
Bonnes pratiques	Verifie multi-stage build, image de base officielle, absence de 'curl bash' piping dangereux.

9.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Acces daemon requis	Necessite l'acces au daemon Docker du serveur pour analyser les images locales.
Images privees	Ne peut analyser les images derriere un registry prive sans credentials explicites.
Analyse statique uniquement	Ne voit pas le comportement a l'execution - un process malveillant au runtime est invisible.
Orchestration non couverte	Ne couvre pas les policies Kubernetes (RBAC) ou Docker Compose multi-services.

9.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Integration Trivy	Remplacer le scanner natif par Trivy (Aqua Security) pour une couverture CVE exhaustive et mise a jour. Priorite Haute.
Registry Scan	Support Docker Hub, ECR et GCR avec credentials chiffres pour les images privees. Priorite Haute.

Runtime Security (Falco)	Coupler avec Falco pour detecter les comportements suspects en production (syscall monitoring). Priorite Moyenne.
Audit Kubernetes	Etendre a l'audit des policies Kubernetes : RBAC, PodSecurityPolicy, NetworkPolicy. Priorite Moyenne.

10. SCANNER DNS & EMAIL

Fichier source : `dns_scanner.py`

Audit de la configuration DNS et de la securite email. Utilise DNS-over-HTTPS (Cloudflare DoH) pour la resolution, verifie SPF/DKIM/DMARC et detecte les sous-domaines exposes via Certificate Transparency.

10.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Enregistrements DNS	A, AAAA, MX, NS, CNAME, TXT - resolution via DNS-over-HTTPS Cloudflare (pas de raw socket requis).
Securite Email	Verifie SPF (anti-relais), DKIM (signature cryptographique) et DMARC (politique de rejet/quarantaine).
Zone Transfer (AXFR)	Tente un transfert de zone pour detecter les serveurs DNS mal configures.
Certificate Transparency	Croise avec crt.sh pour decouvrir des sous-domaines caches lies au certificat.

10.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
DNS-over-HTTPS uniquement	Pas de resolution recursive native - depend de la disponibilite de l'API Cloudflare DoH.
Pas de DNSSEC	Ne verifie pas la signature cryptographique des zones DNS (RRSIG, DS records).
Subdomain takeover passif	Identifie les CNAME suspects mais ne teste pas activement la prise de controle.
Email passif uniquement	Verifie les enregistrements DNS mais n'envoie pas d'email de test pour valider DKIM.

10.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Validation DNSSEC	Verifier la signature cryptographique des zones DNS pour detecter le DNS poisoning. Priorite Haute.
Subdomain Takeover Actif	Detecter les CNAMEs pointant vers des services abandonnes (GitHub Pages, S3, Heroku). Priorite Haute.

DNS History (PassiveDNS)	Comparer DNS actuel vs historique pour detecter des changements suspects recents. Priorite Moyenne.
MTA-STS & BIMl	Verifier MTA-STS (securite transport email) et BIMl (branding email authentifie). Priorite Basse.

11. SCANNER WORDPRESS

Fichier source : [wordpress_scanner.py](#)

Audit de securite dedie aux sites WordPress. Detection de version, audit des plugins et themes, enumeration des utilisateurs via l'API REST et verification des fichiers sensibles exposes.

11.1 Capacites et Perimetre d'Action

Ce que le scanner fait actuellement en production :

Fonctionnalite	Detail de l'analyse
Detection de version	Detecte la version WordPress via meta generator, readme.html et feed RSS.
Audit Plugins/Themes	Liste les plugins actifs et croise avec la WPScan Vulnerability Database.
Enumeration utilisateurs	Teste /wp-json/wp/v2/users pour l'exposition des comptes administrateurs.
Fichiers sensibles	Verifie xmlrpc.php, wp-config.php.bak, debug.log, wp-cron.php exposes publiquement.

11.2 Limites Reelles et Autocritique

Ce que le scanner ne fait PAS et pourquoi :

Limite (Ce qu'il ne fait pas)	Explication technique
Version masquable	Si la version est masquee par un plugin de securite, la precision de l'audit CVE est reduite.
WPScan API limitee	La base CVE WPScan est rate-limited sur le plan gratuit (25 requetes/jour).
Pas de brute force	Ne teste pas les mots de passe - hors scope ethique pour un scanner SaaS.
CMS non WordPress ignores	Ne couvre pas Drupal, Joomla, PrestaShop representant 20% du marche CMS.

11.3 Ameliorations Proposees pour l'Architecture Cible

Axes de R&D concrets pour depasser les limites actuelles :

Axe d'Amelioration	Description Technologique / Impact
Integration WPScan CLI	Integrer WPScan directement pour un scan complet avec base CVE officielle mise a jour quotidiennement. Priorite Haute.
Support Joomla & Drupal	Etendre le scanner CMS a Joomla et Drupal avec leurs propres bases de vulnerabilites. Priorite Haute.
REST API Audit	Scanner tous les endpoints /wp-json/ pour detecter les donnees utilisateurs exposees. Priorite Moyenne.

Login Protection Check	Tester /wp-login.php pour la protection brute force : logout, 2FA, Captcha. Priorite Basse.
------------------------	---